

Clase 265 — Ingeniería inversa de aplicaciones móviles

Parte: **13 — Seguridad móvil, IoT e inalámbrica** · Fuente: *The Mobile Application Hacker's Handbook* (Chell et al.) y documentación de Ghidra/Frida 🛠 Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Dominar la ingeniería inversa (RE) de aplicaciones móviles combinando análisis estático (desensamblado y decompilación de bytecode/nativo) con análisis dinámico (instrumentación con Frida). El alumno recuperará la lógica de una app, entenderá algoritmos ofuscados, localizará y evadirá controles anti-tampering, y parcheará una app para modificar su comportamiento en laboratorio.

⚠ **Nota ética:** el RE se realiza sobre apps propias, de código abierto o CrackMes/retos diseñados para ello, o con autorización. Modificar o redistribuir software de terceros puede infringir derechos de autor y contratos de licencia.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

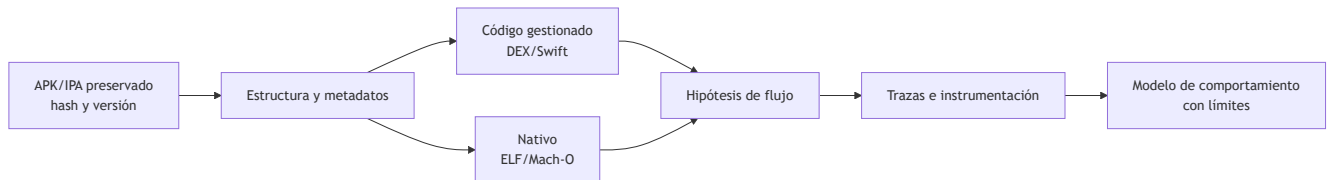
- 1. **Decompilar** APK (smali/Java) y binarios nativos Mach-O/ELF (`.so`).
- 2. **Navegar** código en Ghidra/jadx para reconstruir la lógica de una app.
- 3. **Instrumentar** funciones en runtime con Frida (hooks, tracing, modificación de retornos).
- 4. **Identificar** y evadir ofuscación y anti-tampering (checks de integridad, anti-debug).
- 5. **Parchear** smali o binarios y recompilar/reempaquetar una app propia.
- 6. **Documentar** el proceso de RE de forma reproducible.

📖 Temas

#	Tema	Por qué importa
1	Formatos: DEX, Mach-O, ELF, <code>.so</code>	Determinan la herramienta de RE
2	Desensamblado y decompilación	Recupera lógica legible
3	Análisis dinámico con Frida	Observa el comportamiento real
4	Tracing y hooking	Localiza funciones clave rápido
5	Ofuscación y su análisis	Complica pero no impide el RE
6	Anti-tampering y anti-debug	Controles que hay que evadir
7	Parcheo y reempaquetado	Modificar comportamiento en laboratorio

Reconstruir comportamiento requiere varias representaciones

En Android, APK contiene manifiesto, recursos, DEX y posiblemente bibliotecas ELF; en iOS, el bundle contiene Mach-O, recursos y metadatos. Descompilar produce una aproximación, no el fuente original. Optimización, *inlining*, Swift/Kotlin y ofuscación alteran nombres y estructura. La estrategia alterna evidencia estática con observaciones dinámicas sobre una app propia o expresamente autorizada.



Se comienza por puntos de entrada y flujos de datos, no por leer cada función. Strings, URLs, APIs criptográficas y llamadas nativas orientan; referencias cruzadas revelan quién usa un valor. JNI o bridges conectan capas, de modo que una validación puede estar en Java y una clave derivada en C. La instrumentación confirma argumentos, retornos y estados, pero un hook también puede cambiar tiempos o comportamiento.

La ofuscación eleva coste y reduce señales; no reemplaza proteger secretos ni autorizar en servidor. Reempaquetar una app propia sirve para comprender firma e integridad. Distribuir una app modificada, eliminar controles de licencia o analizar software fuera de permiso puede infringir ley y contrato.

Caso razonado: secreto que no era secreto

JADX muestra una constante llamada `API_KEY`. La traza revela que identifica el cliente ante un servicio público y que toda operación sensible requiere token por usuario. Se reporta exposición de identificador y posible abuso de cuota, no «compromiso total». Otro valor firma solicitudes administrativas y sí cambia impacto. Los nombres orientan; el flujo decide.

Glosario operativo

Término	Definición útil
DEX	Bytecode ejecutado por Android Runtime.
Mach-O	Formato de ejecutables de plataformas Apple.
JNI/bridge	Interfaz entre código gestionado y nativo.
Referencia cruzada	Relación entre uso y definición en un binario.
Reempaquetado	Reconstrucción y firma de una copia autorizada para laboratorio.

Criterio de dominio

El alumno domina RE móvil cuando conserva el artefacto, cruza capas gestionada y nativa, formula hipótesis desde flujos, las valida dinámicamente y calibra el impacto sin confundir nombres u ofuscación con propiedades de seguridad.

Definiciones y características

- **DEX (Dalvik Executable):** bytecode de Android; se desensambla a smali y se decompila a Java aproximado. Característica: legible con jadx.
- **Ghidra:** suite de RE de la NSA para binarios nativos con decompilador a C. Característica: gratuita y potente para ARM/ `.so` /Mach-O.
- **frida-trace:** genera handlers automáticos para funciones y registra sus llamadas. Característica: descubre el flujo sin leer todo el código.
- **Ofuscación:** transformación que dificulta la lectura (renombrado, control-flow flattening, cadenas cifradas). Característica: aumenta el coste, no lo impide.
- **Anti-tampering:** comprobaciones de integridad (firma, checksums) que detectan modificación. Característica: se evaden hookeando el check.
- **Reempaquetado:** recompilar una app modificada y volver a firmarla. Característica: requiere una clave de firma propia.

Herramientas y preparación

```
# Estático
apktool d app.apk -o out          # smali + recursos
jadx-gui app.apk                 # decompilación a Java
# Ghidra para binarios nativos .so / Mach-O

# Dinámico
frida-trace -U -i "recv*" -f <paquete>    # traza funciones que empiezan por recv
frida -U -l script.js -f <paquete>        # inyecta un script propio

# Reempaquetado (app propia)
apktool b out -o mod.apk
apksigner sign --ks mykey.jks mod.apk
```

Laboratorio guiado

1. **Elige un objetivo legítimo:** un CrackMe de Android/iOS o una app propia con un "secreto" a recuperar.
2. **Decompila:** ábrela en jadx y localiza la clase/método que valida el secreto o realiza la lógica de interés.
3. **Analiza el nativo:** si la lógica está en un `.so`, cárgalo en Ghidra y estudia la función exportada relevante.
4. **Instrumenta con Frida:** hooke la función de validación, registra sus argumentos y su valor de retorno.
5. **Modifica el retorno:** cambia el resultado de la función (p. ej. forzar `true`) y observa el cambio de comportamiento.
6. **Evalúa anti-tampering:** si la app comprueba su firma, localiza el check y neutralízalo con un hook.
7. **Parchea estáticamente:** edita el smali correspondiente, recompila con apktool, firma con tu clave e instala la app modificada.
8. **Documenta:** deja un registro reproducible (comandos, offsets, script Frida) del proceso.

Ejercicios

1. Resuelve un CrackMe de Android recuperando el secreto solo con análisis estático.
2. Escribe un script Frida que imprima los argumentos de una función de cifrado.
3. Localiza en Ghidra una función nativa y renómbrala/anota según su propósito.
4. Evade un check de detección de debugger con un hook.
5. Parchea una condición en smali y demuestra el nuevo comportamiento.
6. Compara el esfuerzo de resolver el reto por vía estática vs. dinámica.

Reto verificable

Toma un CrackMe móvil y recupera su secreto/flag por **dos vías independientes**: una estática (leyendo/parcheando el código) y una dinámica (hook con Frida). **Criterio de aceptación**: documentas ambos métodos con evidencia (captura del flag y el script/parche usado) y explicas cuál fue más eficiente y por qué.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
jadx muestra <code>// decompilation failed</code>	Ofuscación agresiva; recurre a smali o análisis dinámico
Frida no engancha la función	Nombre/firma mal escritos o cargada tarde; usa <code>frida-trace</code> para descubrir símbolos
App reempaquetada no instala	No está firmada o firma inválida; fírmala con apksigner
App detecta el parche	Anti-tampering activo; evádelo con un hook antes de parchear
Ghidra no reconoce el <code>.so</code>	Arquitectura equivocada; selecciona el binario ARM correcto

Preguntas frecuentes

 **¿Estático o dinámico: cuál uso primero?** Suelen combinarse. El estático da el mapa general; el dinámico confirma comportamiento y sorteas ofuscación de cadenas y control de flujo.

 **¿La ofuscación hace imposible el RE?** No. Aumenta tiempo y dificultad. La instrumentación puede observar ejecuciones concretas, pero no garantiza cubrir todas las rutas y puede alterar comportamiento; se combina con análisis estático y pruebas de estados distintos.

 **¿Por qué el `.so` requiere Ghidra y el DEX no?** El DEX es bytecode de alto nivel decompilable casi a Java; el `.so` es código máquina ARM/x86 que necesita un decompilador de binarios nativos como Ghidra.

Referencias

- Ghidra: <https://ghidra-sre.org/>
- jadx: <https://github.com/skylot/jadx> · Frida handbook: <https://frida.re/docs/>
- OWASP MASTG — Reverse Engineering: <https://mas.owasp.org/MASTG/0x04c-Tampering-and-Reverse-Engineering/>
- *The Mobile Application Hacker's Handbook*, caps. de RE.



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 264 — Pentest de aplicaciones iOS](#)



Siguiente clase

[Clase 266 — Seguridad de IoT: panorama y superficie de ataque](#)